bcc09 for FLEX Programmers

C for FLEX, for those who know the system

bcc09 project

November 2026

Contents

1 Why bcc09? **1**

2 Your first command **2**

3 The command line **3**

4 The terminal **3**

5 Files **4**

6 Memory **5**

7 FLEX's routines from assembly **5**

8 What's there, and what isn't **7**

9 Testing **7**

Appendix: from FLEX assembly to C **7**

1 Why bcc09?

If you have used FLEX, you know its commands, its `.CMD` files, its File Management System and the DOS routines at `$CD00`, and perhaps wrote your own commands in assembly. *bcc09* is a C compiler for the 6809 that runs on a PC: it compiles a program there, in seconds, into a `.CMD` file that you copy to a FLEX disk and run like any other command.

- **The C is C89,** with prototypes, `void`, structure assignment and the rest, and a C library with `stdio`, `string.h`, `stdlib.h`, `malloc`, and `math.h`, built for FLEX.

- **FLEX is underneath:** the terminal through `INCH` and `PUTCHR` (so `P` and `O` redirect a program's output), files through the FMS, and FLEX's other routines through a few lines of assembly.
- **A command is an ordinary `.CMD` file,** of load records and a transfer address, for wherever in user memory you choose.

This guide is for programmers who know FLEX and some C. It shows how bcc09 builds a command, how a C program meets FLEX, and what to watch for. The bcc09 user's manual has the rest of the compiler and its library.

1.1 What you need

- **bcc09**, on a PC running Linux: on Fedora, the RPM from bcc09's releases on GitHub; otherwise, built from its source.
- **FLEX to run the commands on:** a real machine, or an emulator. bcc09's tests run theirs on Compusense FLEX for the Dragon 64, in XRoar, with the files put on the disk image by a script; any 6809 FLEX takes the same `.CMD` files.
- **sim6809** (`--flex`), which runs a `.CMD` file on your PC with FLEX's routines and its FMS emulated, for trying a command in a moment.

2 Your first command

```c
#include <stdio.h>

int main()
{
    printf("HELLO FROM C\r\n");
    return 0;
}
```

FLEX's terminal starts a new line with a carriage return and a line feed, and bcc09 passes characters as they are, so the program prints `"\r\n"`. Compile it, and make the `.CMD` file:

```
bcc09 -Mf -O -o hello -X-T0x1000 hello.c
objcopy09 -O flex hello HELLO.CMD
```

- **`-Mf`** is the FLEX target: the start-up code and the C library built for FLEX. `-O` runs the peephole optimizer.
- **`-X-T0x1000`** links the command at `$1000`. Choose the address as for any command: in user memory, out of the way of what else will be there (`$1000` is where bcc09's tests put theirs, on the Dragon).
- **`objcopy09 -O flex`** makes the `.CMD` file: load records of up to 255 bytes, and the transfer address.

Copy `HELLO.CMD` to a FLEX disk, and type `HELLO`. `printf` makes it a file of about 4K; `write(1, "HELLO FROM C\r\n", 14)` instead makes it 755 bytes.

2.1 What happens when it runs

FLEX loads the file and jumps to its transfer address, the start-up code, which

- moves the stack from FLEX's own (a small one, below the line buffer) to the end of user memory, `MEMEND`;
- clears the program's uninitialized data (BSS);
- keeps the DP FLEX started it with, to give FLEX's routines when it calls them;
- splits the command line into `argc` and `argv`, and calls `main`.

When `main` returns, or the program calls `exit()`, every file still open is closed (`FMSCLS`) and the program goes back to FLEX through `WARMS`. FLEX has no exit status: the value `main` returns is lost (sim6809 takes it as its own, which tests can check).

The heap, which `malloc()` uses, is the memory between the end of the program and the stack.

3 The command line

```
#include <stdio.h>

int main(argc, argv)
int argc;
char **argv;
{
    int i;

    for (i = 0; i < argc; i++)
        printf("%d: %s\r\n", i, argv[i]);
    return 0;
}
```

- **`argv[0]` is the command's name** as typed: the word before FLEX's line buffer pointer. `P,ARGS` (which prints the output too) gives `ARGS`, as `ARGS` alone does, and `1.ARGS` gives `1.ARGS`.
- **The rest are the words after it,** separated by blanks or commas, as FLEX's own commands take them: `ARGS ONE,TWO THREE` gives `ONE`, `TWO` and `THREE`.

4 The terminal

Output goes through `PUTCHR`, a character at a time, so FLEX's `P` and `O` commands send it to the printer or a file as they would any command's. Input comes from `INCH` a line at a time: `read()` echoes each character through `PUTCHR`, rubs out with TTYSET's backspace character (or control-H), and ends the line at the carriage return, which it shows as CR LF and gives to the program as `'\r'`:

```
#include <stdio.h>
```

```c
int main()
{
    char *line = 0;
    size_t size = 0;
    int n;

    printf("YOUR NAME? ");
    n = getdelim(&line, &size, '\r', stdin);
    if (n > 0) {
        line[n - 1] = 0;    /* (the '\r') */
        printf("HELLO, %s\r\n", line);
    }
    return 0;
}
```

`getline()` and `fgets()` stop at `'\n'`, which a line typed on FLEX doesn't have; `getdelim()` with `'\r'` reads a line. `stdout` is written out at the end of a line, and before input is read, so the prompt, which has no line end, shows before the program waits for the name.

5 Files

`fopen()`, `open()`, `unlink()`, `remove()` and `rename()` go through FLEX's File Management System, with a File Control Block for each open file (up to four at a time). A name is FLEX's: `NAME.EXT`, with a drive before it (`1.NAME.EXT`) or after it (`NAME.EXT.1`); it is made upper case. The name is up to 8 letters, digits, `-` or `_`, starting with a letter, and the extension up to 3; none is added for you.

```c
#include <stdio.h>

int main()
{
    FILE *fp;
    char *line = 0;
    size_t size = 0;
    int n = 0;

    if ((fp = fopen("NOTES.TXT", "w")) == 0) {
        printf("CAN'T WRITE NOTES.TXT\r\n");
        return 1;
    }
    fprintf(fp, "FIRST LINE\r");
    fprintf(fp, "SECOND        LINE\r");
    fclose(fp);

    if ((fp = fopen("NOTES.TXT", "r")) == 0) {
        printf("CAN'T READ NOTES.TXT\r\n");
```

```
        return 1;
    }
    while (getdelim(&line, &size, '\r', fp) > 0) {
        n++;
        printf("%d: %s\n", n, line);    /* (line ends with '\r') */
    }
    fclose(fp);
    return 0;
}
```

FLEX's files have rules of their own, which the library keeps:

- **A file is written from its start, as a new file.** `fopen(name, "w")` (`O_TRUNC`) deletes a file that is there and writes a new one; opening an existing file to write without `O_TRUNC`, to update (`"r+"`, `O_RDWR`) or to add to (`"a"`, `O_APPEND`), fails with `EINVAL`.
- **A text file** has its runs of spaces compressed, as FLEX's editors and `LIST` expect, and its lines end with `'\r'`; reading gives the spaces back.
- **A binary file** (`"wb"`, `"rb"`, or `O_BINARY`) is stored as it is, with no compression. FLEX keeps a file's size in sectors, not bytes, so reading it gives the zeros that pad its last sector too: a program that needs its own data's length should write it at the start.
- **`lseek()`** works on a file open for reading (forward or back: back rewinds and reads again up to the place), not on one being written.

FLEX's errors become `errno`'s `ENOENT` (no such file, or a name FLEX can't take), `EEXIST`, `EBUSY` (in use), `ENOSPC` (the disk is full), `EACCES` (protected), `ENODEV` (no such drive), or, for the rest, `EIO`; a fifth file open at once gives `EMFILE`.

6 Memory

User memory runs from the bottom of memory to `MEMEND` (`$CC2B` holds it): the program is loaded where it was linked, its stack is at the top, and the heap grows up from the end of the program towards it. A command linked at `$1000` on a system with user memory to `$BFFF` has about 44K for its code, its data and the heap together. `malloc()` stops 256 bytes short of the stack pointer, and returns 0 when there is no more.

Large arrays belong in the data (global or `static`), where the map (`-X-M`) shows their size, rather than on the stack.

7 FLEX's routines from assembly

The library calls the routines it needs (`INCH`, `PUTCHR`, the FMS, `WARMS`); for others, write a function in assembly. bcc09's assembler, `as09`, takes Motorola's syntax:

```
; void pstrng(char *s): FLEX's PSTRNG: a new line, then s, which
; ends with an EOT ($04)
```

```
PSTRNG  equ     $CD1E
        .text
        export  _pstrng
        .bss
        import  __sysdp         ; (the start-up code's: FLEX's DP)
        .text
_pstrng:
        pshs    dp,y,u
        lda     __sysdp         ; FLEX's DP, for the call
        tfr     a,dp
        jsr     PSTRNG          ; X: the string, the first argument
        puls    dp,y,u,pc

; void pcrlf(): FLEX's PCRLF: a new line
PCRLF   equ     $CD24
        export  _pcrlf
_pcrlf:
        pshs    dp,y,u
        lda     __sysdp
        tfr     a,dp
        jsr     PCRLF
        puls    dp,y,u,pc
```

- **The first argument is in X,** the rest on the stack above the return address (`2,S`, `4,S` and so on); the result goes back in X.
- **DP must be the same at the end.** A program with `__direct` variables has its own DP while it runs; `__sysdp` keeps the one FLEX started it with, for FLEX's routines, as the library's own calls use.
- **Any other register may change;** saving Y and U, as here, costs little and protects against a routine that changes them.

Assembled with the program (`bcc09 -Mf -O -o greet -X-T0x1000 greet.c pstrng.s`), the functions are called from C:

```c
void pstrng(char *s);
void pcrlf();

int main()
{
    pstrng("HELLO FROM PSTRNG\004");
    pcrlf();
    return 0;
}
```

FLEX's variables, in the DOS's memory, can be read from C without any assembly: `MEMEND` is `*(unsigned *)0xCC2B`.

8 What's there, and what isn't

- **There:** `stdio` (with `printf`'s and `scanf`'s whole formats, floating point among them), `string.h`, `ctype.h`, `stdlib.h` (`malloc`, `qsort`, `strtol`, `rand`), `stdint.h`, `stdbool.h`, `math.h`, and the files above.
- **Not there:** directories (FLEX has none), `time()` (FLEX keeps only a date), `getenv()`, and exit statuses.
- **Floating point** is in software, exact, and optional: a program that doesn't use it doesn't carry it, and one that prints a `double` carries about 6K more for it.

9 Testing

- **sim6809 –flex** runs a `.CMD` file on your PC: `--param` puts words after the command's name in the line buffer, the terminal is stdin and stdout, and the FMS keeps FLEX's files as host files (each the data of its sectors). `WARMS` ends the run, with the value `main` returned as sim6809's exit status.

```
sim6809 --flex --param "ONE,TWO THREE" ARGS.CMD
```

- **FLEX in an emulator** is the real thing: bcc09's own tests boot Compusense FLEX on an emulated Dragon 64, run their commands from a `STARTUP` file with `P,` so that XRoar writes the printer's output to a file, and read the files they write off the disk image, which must be the same, byte for byte, as under sim6809.

Appendix: from FLEX assembly to C

In assembly	In C
`jsr PUTCHR`	`putchar(c)` or `printf(...)`
`jsr PSTRNG`	`printf("\r\n%s", s)`
`jsr PCRLF`	`printf("\r\n")`
`jsr INCH` (a line)	`getdelim(&line, &size, '\r', stdin)`
FMS open for read	`fopen(name, "r")` (`"rb"`: binary)
FMS open for write	`fopen(name, "w")` (`"wb"`: binary)
FMS read a byte	`getc(fp)`
FMS delete	`unlink(name)` or `remove(name)`
FMS rename	`rename(old, new)`
`jmp WARMS`	`return 0;` from `main`, or `exit(0)`
`ldx LPOINT` / `jsr GETFIL…`	`argv[1]`
`rmb 2`	`int x;`
`ldd <x`	`x`, declared `__direct`